16 EXCEPTION HANDLING • THEORY

04 raii noexcept and exception safety guarantees

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

04 — RAII, `noexcept`, and the Three Exception Safety Guarantees

1. Why Raw Pointers Leak During Stack Unwinding — And How RAII Fixes It

Consider what happens when an exception is thrown between `new` and `delete`:

CPP

```cpp
void unsafe_function() {
    int* p = new int[100]; // Allocated on heap
    throw 1;               // Exception thrown!
    delete[] p;            // NEVER EXECUTED! 100 ints leaked!
}
```

During stack unwinding, the runtime destroys local stack variables. Because `p` is a raw pointer (`int*`), destroying `p` simply pops the pointer variable off the stack — it does **not** call `delete[] p` !

Solution: RAII (Resource Acquisition Is Initialization)

Bind the resource to a stack object whose constructor acquires the resource and whose **destructor releases the resource** (such as `std::unique_ptr` , `std::vector` , or `std::fstream`). When stack unwinding destroys the stack object, its destructor automatically frees the heap memory or closes the file!

2. The Three Exception Safety Guarantees (High-Frequency University Exam Topic)

Every function in a C++ program provides one of three formal levels of **Exception Safety** (formulated by David Abrahams):

Guarantee Level	Name	Formal Definition & Guarantees	Example
Level 1	**Basic Exception Guarantee**	If an exception is thrown, **no resources are leaked** and all objects remain in a **valid, self-consistent state** (class invariants are preserved), though the exact values of the objects may have changed partway.	Modifying multiple fields in a container where invariants remain intact
Level 2	**Strong Exception Guarantee (Commit-or-Rollback)**	If an operation throws an exception, the program state is **completely unchanged** (rolled back to the exact state before the function was called — atomic "all-or-nothing" semantics).	Copy-and-swap idiom; `std::vector::push_back()`
Level 3	**No-Throw Guarantee (`noexcept`)**	The operation is guaranteed **never to throw an exception** and always succeeds.	Destructors (`~T()`), `swap()`, move constructors (`noexcept`), simple getters

*(Note: A function that leaks memory or corrupts invariants when an exception occurs offers **No Exception Guarantee**.)*

---

3. The `noexcept` Specifier and Destructor Rule

CPP

```cpp
int get_balance() const noexcept {
    return balance;
}
```

1. Marking a function `noexcept` tells both the human reader and the C++ compiler that the function guarantees not to throw any exceptions.
2. **What happens if a `noexcept` function throws anyway?**: The runtime does **not** propagate the exception to callers; it immediately calls `std::terminate()`.
3. **Destructors are implicitly `noexcept` since C++11**: Never let an exception escape from a destructor! If a destructor throws an exception while stack unwinding is already in progress from another exception, C++ immediately calls `std::terminate()`.